

Geo-referenced Photo Management System

Presentation for the Context Aware Systems Project

Gabriele Spina

Università degli Studi di Bologna
Laurea Magistrale in Informatica



Introduction

Photos Geo-Localization

- The geo-localization of the photos is a functional information
- The usage of geo-localization in the construction of a photo management system can significantly improve the quality of the service offered

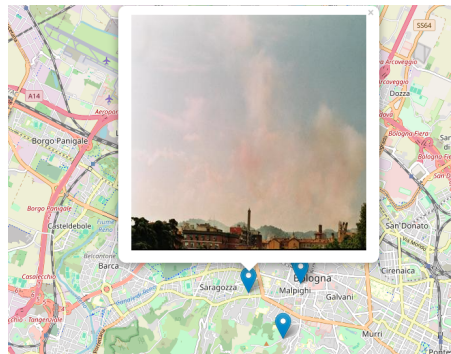


Figure 1: Example: visualize photos as markers on the map with the possibility to click on the marker to see the related photo

- In this work a geo-referenced photo management system is constructed
- A spatial database is implemented to store and retrieve the data
- There is a mobile application from which the user can take photos and visualize them
- The feature of visualizing photos on the map, as in the image in the previous slide, is implemented in a web dashboard

Spatial Database

- The spatial database is implemented using the PostgreSQL database management system
- The database is composed of two tables: one for photos and one for the referring collection

Media Files

id	serial
name	varchar
date	timestamp
image	bytea
position	geography(point)
collection_id	int

Table 1: Media files table design.

Collections

collection_id	serial
name	varchar
created_at	timestamp

Table 2: Collections table design.

Mobile Application

- The application is a simple gallery, from which images in different collections can be visualized
- Implemented using Kivy library with Python language
- There is the possibility to take a new photo and save it in a collection
- Then, other features, like the creation of a new collection or the retrieval of the photos taken nearest to the user position

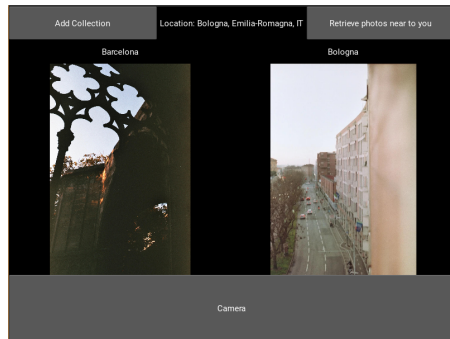


Figure 2: Mobile Gallery, main screen

Web Dashboard

With the mobile application there is not the possibility to visualize photos on map, or even to upload more photos at once; a web dashboard is constructed to overcome these issues and provide more features.

- The frontend is implemented with Leaflet, JavaScript, HTML; the data received from the backend is visualized on the map on the basis of user's actions
- The backend is implemented with Flask and Python; the requests received from the frontend are satisfied by querying the database and manipulating data, according to the particular request

Web Dashboard Features

The web dashboard (on the right) offers the possibility to interact with photos in different ways:

- **Photo Upload:** single photo or folder, with the possibility to apply spatial perturbation to the saving position for folder case
- **GeoJson Upload:** upload a file containing zones definition, filter photos by zones
- **Cluster and HeatMap:** cluster all photos in groups, visualize photos in a heatmap
- **Chronological Path:** build a path reconstructing the road crossed from various photos
- **Gallery:** visualize all collections and photos

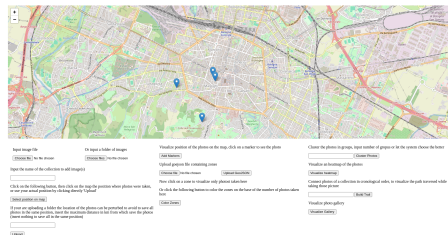


Figure 3: Web Dashboard, main screen

Conclusion

- The system works well in a local environment with a small amount of data.
- The scalability of the system with large amounts of data is not so good, mainly because the photos are stored directly in the database
- To improve that, photos could be stored in a cloud and then referred in the database with the link

Thanks for your attention!

Now a demo of the system...

